

Beatriz São Leandro Cosimatti
17024130

João Victor Bozola Bosi
5979703

João Vitor Vasconcelos Jacob
16987258

Laís Antunes Cayres Quintão
16880392

Trabalho de Estrutura de Dados II:
Análise e Implementação de Algoritmos

Disciplina: Estrutura de Dados II
SCC0606
Professor: Careca safado

São Carlos
14 de Junho de 2026

1 Introdução

O objetivo do trabalho desenvolvido foi estudar e implementar diferentes algoritmos de ordenação na linguagem C. Como cada algoritmo se comporta de uma maneira específica dependendo do tamanho do vetor ou de como os números estão organizados, o objetivo final do projeto é a criação de um Sistema Adaptativo. Esse sistema funciona como um analisador que avalia o vetor de entrada e, através de heurísticas, escolhe automaticamente o melhor algoritmo para realizar a ordenação.

Para fundamentar as tomadas de decisão desse sistema, o projeto foi estruturado de maneira modular, englobando a implementação de cinco algoritmos: Bubble Sort, Insertion Sort, Quick Sort, Heap Sort e Radix Sort.

ACHO Q COLOCA O LINK DO VIDEO AQ

2 Algoritmos Base

2.1 Bubble Sort

O Bubble Sort (Ordenação por Bolha) é um algoritmo de ordenação baseado no princípio de comparações e trocas adjacentes. Ele percorre o vetor repetidamente a partir do início comparando pares de elementos vizinhos (vetor[i] e vetor [i+1]), e troca-os de lugar se o elemento da esquerda for maior que o da direita. Esse processo é repetido até que nenhuma troca seja necessária em uma passada completa, indicando que o conjunto de dados está totalmente ordenado, sendo essa a versão aprimorada do Bubble Sort.

Dessa forma, a sua complexidade de tempo é $O(n^2)$, exceto no caso em que o vetor já está ordenado. Nessa situação, o vetor só é percorrido uma única vez, tornando a complexidade $O(n)$. É um algoritmo estável e in-place, mas seu comportamento degrada-se drasticamente em entradas grandes devido ao crescimento quadrático das operações de comparação e movimentação de dados.

2.2 Insertion Sort

O Insertion Sort (Ordenação por Inserção) consiste em inserir um novo elemento em um vetor já ordenado ou quase ordenado. Ele percorre o vetor a partir do segundo elemento (índice 1), selecionando-o como uma "chave", e compara essa chave com os elementos que estão nas posições anteriores (à sua esquerda). Caso os elementos anteriores sejam maiores que a chave, eles são deslocados uma posição para a direita para abrir espaço, e esse processo se repete até encontrar a posição correta, onde a chave é inserida. Ele percorre o vetor a partir do segundo elemento (índice 1), selecionando-o como uma "chave", e compara essa chave com os elementos que estão nas posições anteriores (à sua esquerda). Caso os elementos anteriores sejam maiores que a chave, eles são deslocados uma posição para a direita para abrir espaço, e esse processo se repete até encontrar a posição correta, onde a chave é finalmente inserida.

Assim, sua complexidade de tempo é $O(n^2)$, exceto no caso em que o vetor já está ordenado (ou quase ordenado). Nessa situação, a chave é comparada apenas uma vez com o elemento imediatamente anterior e o loop interno é interrompido por um break, tornando a complexidade $O(n)$. É um algoritmo estável e in-place, mas seu comportamento degrada-se drasticamente em entradas grandes e aleatórias.

2.3 Quick Sort

O algoritmo de ordenação Quick Sort foi desenvolvido e publicado pela primeira vez em 1961, pelo cientista da computação britânico Charles Antony Richard Hoare (1934 - 2026). A técnica de ordenação foi concebida enquanto o cientista estudava na Universidade de Moscou e trabalhava no *National Physical Laboratory (NPL)*, do Reino Unido. Na ocasião, Hoare estava participando de um projeto que tinha como objetivo a tradução de textos em russo para inglês e, para isso, como os dicionários eram armazenados em fita magnética, necessitava ordenar as palavras dentro de uma sentença em ordem alfabética. Nesse contexto, surgiu a necessidade de desenvolver um algoritmo de ordenação que fosse assintoticamente menor que $O(n^2)$.

Sob essa ótica, o algoritmo é baseado no paradigma da divisão e conquista. Nesse sentido, a aplicação se consiste em ordenar o vetor em torno de um pivô escolhido, de modo que, os elementos à esquerda do pivô sejam menores que ele e os elementos à direita, consequentemente, sejam maiores. Dessa forma, o vetor é subdividido em duas partições, que terão o mesmo processo aplicado sobre elas, através de chamadas recursivas da função. Assim, as chamadas se encerram quando o caso base de um elemento na partição é atingido, uma vez que ele já estará ordenado.

Tais comportamentos destacados são os aspectos base do algoritmo quick sort, no entanto, cada aplicação dista da técnica utilizada para escolher o elemento pivô e para como o particionamento será realizado. Para o algoritmo implementado neste projeto, utilizou-se o método da mediana de 3 para se determinar o pivô. Ele se consiste em armazenar o primeiro, central e último elemento do vetor em variáveis auxiliares e retornar a mediana entre eles, nesse caso, o elemento do meio. Essa técnica, embora seja mais complexa do que aplicações comuns que utilizam o primeiro ou último elemento do array como pivôs, tem como benefícios a melhora do tempo de execução e a possibilidade da execução possuir uma complexidade de $O(n \log n)$ mesmo para vetores ordenados ou parcialmente ordenados - embora as métricas de análise dos vetores utilizadas para esse projeto evitem tais casos.

No caso da partição, existem três principais métodos comumente utilizados para o quick sort, o particionamento de Lomoto, Naive e Hoare, para este trabalho se utilizou o particionamento de Hoare. O método se consiste em na definição de dois ponteiros, nas duas extremidades do vetor e que, a cada interação, se movem em direção de seu encontro até que um elemento que necessita ser trocado de lugar seja encontrado. Considerando i o elemento na posição zero do vetor e j o elemento na posição final ambos são movidos até que i encontre um elemento maior que o pivô e, analogamente, j encontre um elemento menor que o pivô. Nesse caso, os valores encontrados trocam de posição e i e j voltam a percorrer o vetor até que eles se cruzem ou sobreponham. Destaca-se, que assim como a partição de Lomoto, o algoritmo de Hoare possui uma complexidade de tempo de $O(n)$ e de espaço de $O(1)$, contudo, ela é considerada mais rápida que o outro método, uma vez que, no geral, realiza menos trocas entre elementos. Em relação ao particionamento de Naive, o algoritmo se baseia na criação de um vetor auxiliar, ou seja, a complexidade de espaço é de $O(n)$, sendo significativamente menos eficiente que o método utilizado.

Dado as características da partição de Hoare, é importante destacar que o quick sort é um algoritmo *in-place*, ou seja, não utiliza nenhum vetor auxiliar para a ordenação, uma vez que as trocas são realizadas dentro do próprio vetor. Além disso, o algoritmo não é estável, uma vez que não preserva a ordem relativa entre elementos de mesmo valor. É também importante destacar que o quick sort apresenta um comportamento muito eficiente em termos de acesso à memória devido à forma como realiza o particionamento do vetor. Durante essa etapa, os elementos são percorridos sequencialmente por meio de índices que avançam a partir das extremidades do arranjo, favorecendo a localidade espacial dos dados. Como elementos próximos na memória tendem a ser acessados em instantes próximos, a hierarquia de memória do processador, especialmente os níveis de cache, é utilizada de maneira mais eficiente. Além disso, à medida que as partições são realizadas, o algoritmo passa a operar sobre subvetores progressivamente menores, os quais frequentemente passam a caber integralmente na memória cache. Dessa forma, reduz-se a quantidade de acessos à memória principal, resultando em uma diminuição da latência de acesso aos dados e contribuindo significativamente para o elevado desempenho prático do algoritmo.

Ademais, é possível analisar o melhor, pior e médio caso do quick sort. Nota-se que o melhor caso ocorre quando o pivô selecionado é exatamente o elemento central do vetor analisado.

$$T(n) = 2T\left(\frac{n}{2}\right) + n$$

Pela análise da recursão, através do Teorema Mestre, obtém-se a complexidade de espaço para o melhor caso como $O(n \log n)$. Nesse sentido, para o médio caso, considera-se que o array é dividido em duas partes de tamanho K , sendo assim:

$$T(n) = T(n - K) + T(n)$$

Após a resolução da recorrência, também nota-se que a complexidade é de $O(n \log n)$. Nesse sentido, evidencia-se que no melhor e médio caso, o quick sort utiliza em média $2 n \log n$ comparações entre elementos distintos do vetor para ordená-lo, e aproximadamente, um sexto desse número de trocas. Por fim, o pior caso ocorre quando o pivô escolhido é sempre o menor ou o maior elemento do subvetor analisado. Nessa situação, a cada etapa do algoritmo são geradas partições extremamente desbalanceadas, contendo 0 e $n - 1$ elementos, o que resulta em uma árvore de recursão linear e leva a uma complexidade temporal de $O(n^2)$:

$$T(n) = T(n - 1) + n$$

Para o pior caso, observa-se que o algoritmo necessita realizar, em média, $n^2/2$ comparações para ordenar o vetor, o que representa um aumento assintótico significativo em relação aos outros casos. Em relação à complexidade de espaço, cada chamada recursiva apresenta complexidade $O(1)$ - devido a característica *in-place* do algoritmo -, no entanto, múltiplas chamadas ocorrem simultaneamente, modificando o comportamento da complexidade espacial conforme o caso de execução. Para o médio e melhor caso, a árvore de recursão gerada é balanceada e possui profundidade de $O(\log n)$, o que acarreta em uma complexidade de espaço de $O(\log n)$. Em contrapartida, devido ao comportamento linear da árvore no pior caso, sua altura é de $O(n)$, acarretando em uma complexidade assintótica de $O(n)$. Assim, foi possível elaborar a seguinte tabela comparativa:

Varição	Complexidade de tempo	Complexidade de espaço
Melhor caso	$O(n \log n)$	$O(\log n)$
Caso médio	$O(n \log n)$	$O(\log n)$
Pior caso	$O(n^2)$	$O(n)$

Tabela 1: Complexidade de tempo e espaço do quick sort.

2.4 Heap Sort

O Heap Sort baseia-se na estrutura de dados *Max-Heap*, uma árvore binária quase completa mapeada implicitamente em um vetor. Para um nó no índice i , seus filhos estão localizados em $2i + 1$ e $2i + 2$, e o valor do pai é sempre maior ou igual ao de seus filhos.

A ordenação ocorre em duas etapas: a construção do heap (**Build-Max-Heap**), que reorganiza o vetor em tempo linear, e a extração sucessiva do elemento máximo. A cada extração, o último elemento do subvetor substitui a raiz, e a função **Max-Heapify** restaura a propriedade da árvore.

Matematicamente, a etapa de construção possui custo assintótico linear:

$$O\left(\sum_{h=0}^{\lfloor \log_2 n \rfloor} \frac{n}{2^{h+1}} h\right) = O(n)$$

A segunda etapa percorre a altura da árvore para $n - 1$ extrações, custando $O(\log_2 n)$ cada. Assim, a complexidade total resulta em:

$$T(n) = O(n) + O(n \log_2 n) = O(n \log_2 n)$$

Isso garante ao Heap Sort um desempenho de $O(n \log_2 n)$ em qualquer cenário, blindando-o contra piores casos. Além disso, é um algoritmo estritamente *in-place*, com complexidade espacial de $O(1)$.

Apesar da estabilidade matemática, o Heap Sort sofre em hardwares modernos devido à sua péssima localidade espacial de referência. A navegação não-sequencial pelos índices da árvore (i para $2i + 1$ e $2i + 2$) esgota rapidamente a capacidade da memória Cache (L2/L3). Em vetores massivos, isso gera uma alta taxa de *Cache Misses*, forçando a CPU a paralisar seus ciclos para buscar dados na memória RAM, o que degrada o seu desempenho prático.

Varição	Complexidade de tempo	Complexidade de espaço
Melhor caso	$O(n \log_2 n)$	$O(1)$
Caso médio	$O(n \log_2 n)$	$O(1)$
Pior caso	$O(n \log_2 n)$	$O(1)$

Tabela 2: Complexidade de tempo e espaço do Heap Sort.

2.5 Radix Sort

O Radix Sort é um algoritmo não-comparativo que supera o limite estrutural de $\Omega(n \log_2 n)$ operando sobre os dígitos individuais dos elementos. Em sua variante *Least Significant Digit* (LSD), o processamento ocorre da direita para a esquerda (das unidades para as casas maiores). Para isso, o método exige obrigatoriamente um algoritmo auxiliar estável, tipicamente o Counting Sort.

A aplicação identifica o maior elemento para determinar o número máximo de dígitos (d). Em seguida, o Counting Sort é invocado d vezes para agrupar e reordenar os elementos em cada casa decimal. Sendo n o tamanho do vetor e k a base numérica, cada rodada custa $O(n + k)$, resultando na complexidade de tempo:

$$T(n) = O(d \cdot (n + k))$$

Para tipos de dados nativos onde d e k são tratados como constantes, o comportamento assintótico simplifica-se para um modelo linear de $O(n)$.

Entretanto, essa eficiência temporal cobra um alto preço em hardware. O Radix Sort não é *in-place*, exigindo a alocação de vetores auxiliares com complexidade espacial de $O(n + k)$. Em conjuntos de dados massivos, essa alocação pode exaurir rapidamente a memória RAM do sistema, forçando a paginação no disco de armazenamento (HD/SSD) e elevando a latência de forma drástica. Adicionalmente, o tratamento de números negativos exige rotinas extras de *offsetting* (deslocamento numérico unificado) para evitar falhas de acesso, adicionando sobrecarga ao algoritmo inicial.

Variação	Complexidade de tempo	Complexidade de espaço
Melhor caso	$O(d \cdot (n + k))$	$O(n + k)$
Caso médio	$O(d \cdot (n + k))$	$O(n + k)$
Pior caso	$O(d \cdot (n + k))$	$O(n + k)$

Tabela 3: Complexidade de tempo e espaço do Radix Sort.

3 Arquitetura do Sistema

O sistema adaptativo desenvolvido tem como objetivo selecionar automaticamente o algoritmo de ordenação mais adequado para cada vetor de entrada. Para isso, antes da ordenação, são coletadas diversas métricas que descrevem as características dos dados. A partir dessas informações, um conjunto de heurísticas decide qual algoritmo apresenta maior potencial de desempenho para a entrada analisada.

3.1 Métricas Coletadas

3.1.1 Grau de Desordem

O grau de desordem mede o percentual de inversões adjacentes existentes no vetor. Uma inversão adjacente ocorre quando um elemento é maior que seu sucessor imediato.

A métrica é calculada por:

$$Desordem = \frac{\text{Quantidade de Inversões Adjacentes}}{n - 1} \times 100$$

Essa métrica foi utilizada para identificar vetores quase ordenados, situação na qual o Insertion Sort apresenta desempenho próximo de $O(n)$.

3.1.2 Quantidade de Elementos Repetidos

Esta métrica contabiliza quantos elementos do vetor são cópias extras de valores já existentes.

Seu objetivo inicial era investigar se a presença de muitos valores repetidos poderia influenciar a escolha do algoritmo de ordenação.

3.1.3 Amplitude dos Valores (Range)

A amplitude dos valores corresponde à diferença entre o maior e o menor elemento do vetor:

$$Range = Max - Min$$

Essa métrica foi utilizada para avaliar a adequação do vetor à aplicação do Radix Sort.

3.1.4 Desvio Padrão

O desvio padrão mede o grau de dispersão dos elementos em relação à média do conjunto de dados.

$$\sigma = \sqrt{\frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n}}$$

Inicialmente investigou-se a possibilidade de utilizar essa métrica como parâmetro para seleção do algoritmo.

3.1.5 Valores Máximo e Mínimo

Além de serem utilizados para o cálculo da amplitude dos dados, os valores máximo e mínimo permitem identificar a presença de números negativos, informação importante para a escolha do algoritmo de ordenação.

3.2 Seleção das Métricas

Embora todas as métricas tenham sido implementadas e avaliadas experimentalmente, apenas aquelas que demonstraram influência consistente no desempenho dos algoritmos foram incorporadas à heurística final do sistema.

Tabela 4: Métricas implementadas e utilização na heurística final

Métrica	Utilizada na heurística
Grau de desordem	Sim
Quantidade de repetidos	Não
Amplitude (Range)	Sim
Desvio padrão	Não
Valor mínimo	Sim

A quantidade de elementos repetidos e o desvio padrão foram mantidos apenas para fins de caracterização dos conjuntos de dados. Durante os experimentos realizados, essas métricas não apresentaram influência significativa na escolha do algoritmo mais eficiente.

3.3 Heurística Final

Após a realização dos experimentos, a seguinte estratégia foi adotada:

```

if (n <= 1000 && desordem <= 5.0)
    return "insertion";

if (min < 0 && n >= 10000)
    return "heap";

if (min >= 0 && range <= 10*n)
    return "radix";

return "quick";

```

A lógica empregada pode ser resumida da seguinte forma:

- Vetores pequenos e quase ordenados são processados pelo Insertion Sort.
- Vetores contendo valores negativos podem ser direcionados ao Heap Sort.
- Vetores cuja amplitude seja proporcional ao tamanho são direcionados ao Radix Sort.
- Os demais casos são tratados pelo Quick Sort.

A regra associada à presença de valores negativos foi incluída por razões teóricas relacionadas à aplicabilidade do Radix Sort.

Os conjuntos de dados utilizados durante os experimentos continham apenas valores não negativos, impossibilitando a validação experimental dessa decisão. Dessa forma, a utilização do Heap Sort para vetores negativos de grande porte foi baseada em propriedades teóricas dos algoritmos analisados e deve ser entendida como uma decisão conservadora da heurística.

4 Validação das Heurísticas

A definição da heurística final foi baseada em uma série de hipóteses experimentais avaliadas durante o desenvolvimento do projeto.

4.1 Hipótese 1: Vetores Pequenos Favorecem o Insertion Sort

Inicialmente assumiu-se que vetores com menos de 50 elementos deveriam ser processados pelo Insertion Sort.

Para validar essa hipótese, foram executados experimentos utilizando vetores aleatórios de tamanho 50 e comparando os resultados obtidos por todos os algoritmos implementados.

Os resultados mostraram que o Quick Sort apresentou desempenho superior ao Insertion Sort mesmo para esse tamanho de entrada.

Dessa forma, a regra:


```
if (n <= 50)
    return "insertion";
```

foi removida da heurística final.

Conclusão: hipótese rejeitada.

4.2 Hipótese 2: Vetores Quase Ordenados Favorecem o Insertion Sort

Foram realizados experimentos com vetores quase ordenados contendo níveis crescentes de desordem.

Observou-se que o Insertion Sort apresentou desempenho significativamente superior para vetores com grau de desordem reduzido, mantendo comportamento próximo ao caso linear esperado pela teoria.

A heurística baseada em:

```
n <= 1000 && desordem <= 5
```

mostrou-se consistente durante todos os experimentos realizados.

Conclusão: hipótese confirmada.

É importante destacar que vetores ordenados ou quase invertidos normalmente representam entradas problemáticas para implementações simples do Quick Sort que utilizam o primeiro ou o último elemento como pivô. Entretanto, a implementação desenvolvida neste trabalho utiliza a estratégia da mediana de três para seleção do pivô, considerando os elementos inicial, central e final do subvetor.

Essa abordagem reduz significativamente a probabilidade de partições extremamente desbalanceadas em vetores parcialmente ordenados ou invertidos. Como consequência, observou-se experimentalmente que o Quick Sort manteve desempenho compatível com o comportamento esperado de $O(n \log n)$ nesses cenários.

4.3 Hipótese 3: O Radix Sort Necessita de um Tamanho Mínimo

Inicialmente foi considerada a utilização de um limite inferior para ativação do Radix Sort.

Entretanto, testes realizados com vetores de aproximadamente 5000 elementos mostraram que o algoritmo já apresentava desempenho superior ao Quick Sort.

Dessa forma, o limite mínimo inicialmente considerado foi removido.

Conclusão: hipótese rejeitada.

4.4 Hipótese 4: Vetores Grandes Favorecem o Radix Sort

Experimentos realizados com vetores aleatórios de 10000 elementos confirmaram que o Radix Sort apresenta desempenho superior quando a amplitude dos valores é compatível com o tamanho do vetor.

Conclusão: hipótese confirmada.

4.5 Hipótese 5: Vetores Discrepantes Devem Evitar o Radix Sort

Considerou-se inicialmente que vetores com amplitudes extremamente elevadas deveriam evitar o uso do Radix Sort.

Os resultados mostraram que, mesmo em cenários discrepantes, o Radix Sort continuou apresentando desempenho competitivo. Entretanto, observou-se que o aumento da amplitude dos dados pode tornar o Quick Sort uma alternativa interessante devido ao menor consumo de memória auxiliar.

Dessa forma, conclui-se que a amplitude dos dados deve ser considerada pela heurística, porém não foi observada uma degradação suficientemente significativa que justificasse a exclusão completa do Radix Sort nesses cenários.

Conclusão: hipótese parcialmente aceita.

4.6 Hipótese 6: Vetores Muito Grandes Favorecem o Heap Sort

Também foi investigada a possibilidade de utilizar o Heap Sort para vetores extremamente grandes. Foram realizados experimentos com vetores contendo até 500000 elementos.

Os resultados mostraram que o Heap Sort permaneceu consistentemente inferior ao Quick Sort e ao Radix Sort em termos de tempo de execução e número total de operações.

Consequentemente, não foi identificada uma situação experimental em que sua utilização apresentasse vantagens claras.

Conclusão: hipótese rejeitada.

5 Metodologia Experimental

Descrever: • ambiente utilizado; • datasets; • tamanhos testados; • critérios de avaliação

6 Resultados

Apresentar tabelas, gráficos, métricas coletadas, com discussão crítica. Explicar: • decisões corretas; • decisões ruins; • comportamentos inesperados; • limitações observadas; • impacto das características da entrada. • impactos das entradas adversariais

6.1 Quick Sort

Com o intuito de validar a algoritmo quick sort implementado neste projeto, analisou-se as curvas do número de comparações entre elementos do vetor e número de trocas necessárias para atingir a ordenação - para valores com diferentes ordens de grandeza - quais foram estudadas teoricamente e possuem comportamento esperado definido. Dessa forma, nota-se que o número de comparações do quick sort para o melhor caso é na ordem de $2 n \log n$, e um sexto desse número para a quantidade de trocas.

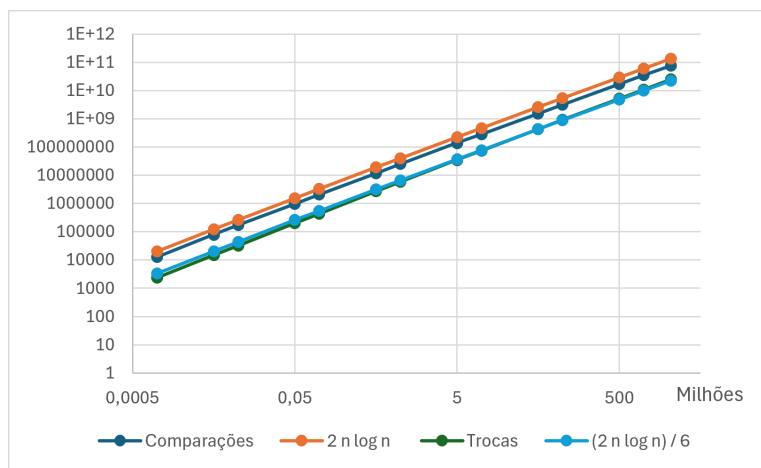


Figura 1: Comparativo entre o número de comparações e trocas e seu comportamento esperado para o melhor caso do quick sort. Escala dilog.

No gráfico é possível observar que o comportamento esperado é efetivo, uma vez que os dados coletados acompanham as linhas das funções que descrevem o comportamento previsto. Observa-se que o gráfico foi elaborado em escala log-log para a melhor visualização do comportamento para as diferentes ordens de grandeza.

Na implementação do quick sort desenvolvida, o pivô é escolhido por meio do método da mediana de três, utilizando os elementos inicial, central e final do subvetor. Essa estratégia reduz significativamente a ocorrência de partições desbalanceadas, especialmente em vetores já ordenados ou parcialmente ordenados. Ainda assim, o pior caso permanece teoricamente possível e ocorre quando a mediana desses três elementos produz repetidamente pivôs próximos aos extremos do subvetor, gerando partições de tamanhos aproximadamente 0 e $n - 1$. Nesse sentido, o método da mediana de três foi amplamente estudada por cientistas como David Musser, pesquisador conhecido por desenvolver o algoritmo Introsort, atualmente utilizado em diversas bibliotecas padrão da linguagem C++. Em seu trabalho, Musser demonstrou que, embora a estratégia da mediana de três reduza significativamente a probabilidade de partições desbalanceadas em entradas comuns, ela não elimina completamente o pior caso do Quick Sort. Sob essa ótica, existem famílias de sequências especialmente construídas, conhecidas como *Median-of-Three Killer Sequences*, capazes de induzir a escolha repetida de pivôs inadequados, resultando em partições altamente desbalanceadas e levando o algoritmo a uma complexidade temporal quadrática $O(n^2)$.

Algumas dessas sequências conhecidas são a *organ pipe* e intercalada. A sequência *organ pipe* se consiste em um array com o elemento central possuindo o maior valor do vetor e os elementos a esquerda e a direita dele decrescendo. Um exemplo de uma distribuição dessa forma, contínua, para $n = 10$ é: $[1 \ 2 \ 3 \ 4 \ 5 \ 6 \ 4 \ 3 \ 2 \ 1]$. Em relação a distribuição intercalada, o vetor se consiste em valores intercalados entre uma sequência crescente e outra decrescente, dessa forma, para $n = 10$, uma possível

configuração seria $[1\ 10\ 2\ 9\ 3\ 8\ 4\ 7\ 5\ 6]$. No gráfico a seguir, é possível visualizar um comparativo entre os tempos de execução para vetores de distribuição aleatória e distribuição seguindo o padrão *organ pipe*:

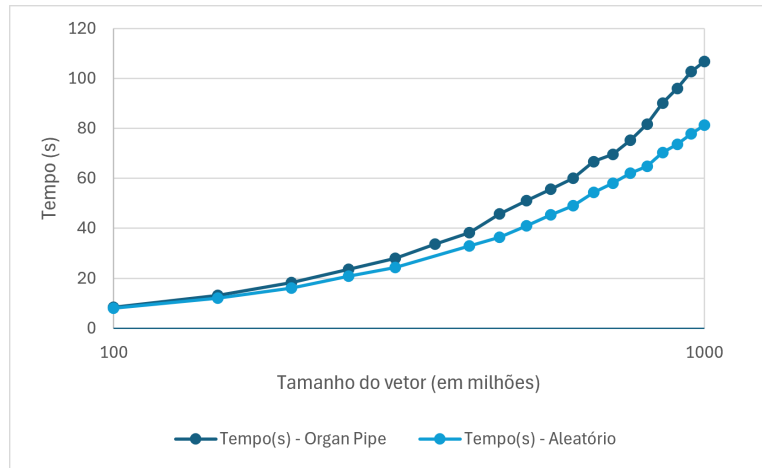


Figura 2: Comparatio de tempo de execução para a ordenação quick sort entre vetores de distribuição aleatória e quick sort.

Através dos dados, é possível observar que essa distribuição não stressou devidamente o algoritmo, uma vez que a curva de ambos apresentou uma complexidade temporal de $O(n \log n)$, embora a dsitribuição *organ pipe* tenha apresentado tempos de execução ligeiramente maiores. É válido destacar o comportamento da métrica que mede a profundidade máxima árvore de rescursão para ambos os casos.

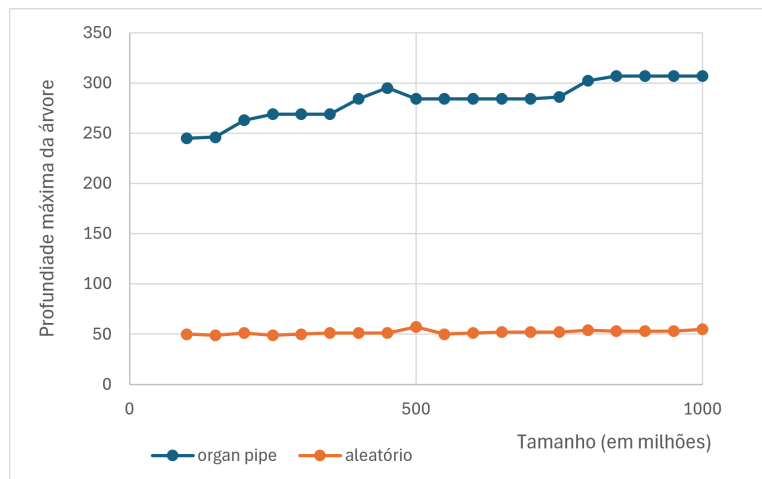


Figura 3: Comparatio de tempo de execução para a ordenação quick sort entre vetores de distribuição aleatória e quick sort.

Observa-se que a profundidade máxima para a distribuição *organ pipe* foi significativamente maior do que para o caso aleatório, indicando que a escolha do pivô para esses vetores não foi tão eficaz quanto o observado no melhor caso, o que acarretou na necessidade de um maior número de partições e, consequentemente, uma maior árvore recursiva. Assim, os arrays criandos seguindo o modelo *organ pipe* não stressaram o algoritmo implementado o suficiente para prejudicar a sua complexidade assintótica de tempo para $O(n^2)$, à medida que, como observado e analisado, a ditribuição se enquadra em um médio caso do quick sort.

Em relação aos vetores intercalados, o quick sort não conseguiu ordenar arrays com mais de 100 milhões de tamanho, uma vez que a péssima escolha dos pivôs para esse caso acarretaram em árvores

de recursão extremamente grandes - com profundidade máxima maior que 80.000 para vetores na ordem de 200 milhões de tamanho - o que ocasionou a interrupção prematura da ordenação devido ao esgotamento da pilha de chamadas. Dessa forma, observa-se que o pior caso esperado foi atingido, acarretando em uma execução na ordem de $O(n^2)$ e incapaz de ser finalizada para vetores grandes, como os analisados. Nesse sentido, evidencia-se também que para vetores menores aos analisados, de ordem de grandeza 10.000, a profundidade máxima da recursão atingiu valores muito maiores do que a ordenação de vetores aleatórios na casa de 100 milhões de tamanho - por exemplo, um vetor aleatório de tamanho 1 bilhão atingiu profundidade máxima de 55, enquanto um vetor intercalado de 100 mil, atingiu 507. Esses comportamentos destacam que a estratégia da mediana de três, embora eficiente em casos usuais, permanece suscetível a entradas adversárias capazes de produzir partições fortemente desbalanceadas.

6.2 Heap Sort

Para ilustrar esse comportamento assintótico teórico de $O(n \log_2 n)$, isolado dos ruídos causados pela infraestrutura física da máquina, elaborou-se o gráfico a seguir. Ele cruza o crescimento do tamanho do vetor exclusivamente com o número de operações matemáticas (comparações e trocas):

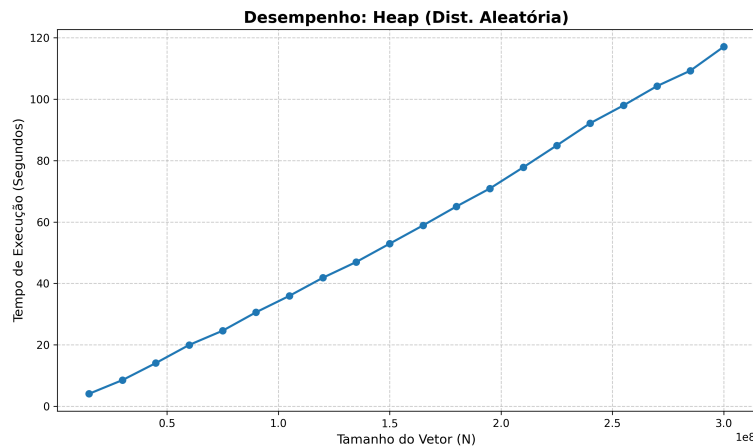


Figura 4: Curva assintótica evidenciando a complexidade constante de $O(n \log_2 n)$ do Heap Sort baseada no número de comparações lógicas.

Apesar de sua elegância e previsibilidade matemática, o Heap Sort apresenta um severo gargalo arquitetural em hardwares modernos devido à sua **baixa localidade espacial de referência**. A navegação entre nós pais (índice i) e filhos (índices $2i + 1$ e $2i + 2$) gera saltos de memória distantes e não sequenciais.

Quando o vetor extrapola a capacidade de armazenamento da memória Cache (níveis L2 ou L3) do processador, esses saltos causam uma incidência massiva de *Cache Misses*. A CPU é forçada a paralisar seus ciclos enquanto aguarda a lenta busca de dados na memória RAM, o que degrada substancialmente seu desempenho prático. Em contrapartida, algoritmos como o Quick Sort operam de forma sequencial ($i++$ e $j--$), maximizando os acertos na Cache. Esse estresse físico na hierarquia de memória é evidenciado no comparativo de execução a seguir:

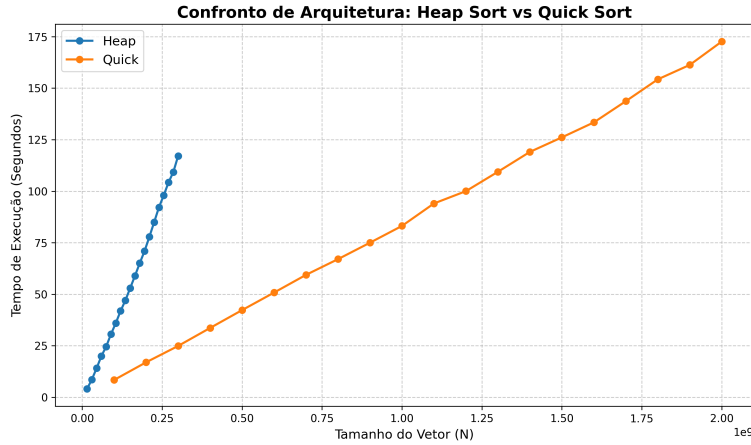


Figura 5: Comparativo de tempo de execução entre Heap Sort e Quick Sort evidenciando a degradação temporal do Heap Sort decorrente de Cache Misses para vetores massivos.

6.3 Radix Sort

Com o intuito de validar a implementação e o comportamento do Radix Sort, a análise de resultados focou em sua característica de ordenação não-comparativa e em sua intensa demanda por recursos de memória. Ao contrário do Quick Sort e do Heap Sort, que estão limitados matematicamente pela barreira inferior de $\Omega(n \log_2 n)$, o Radix Sort ordena os dados processando dígitos individuais através de um algoritmo estável auxiliar (neste caso, a abordagem *Least Significant Digit* - LSD, utilizando o Counting Sort). Sob essa ótica, o algoritmo atinge uma complexidade temporal linear teórica de $O(d \cdot (n + k))$. Como os testes experimentais foram realizados com números inteiros nativos de limites conhecidos, o número máximo de dígitos d e a base de contagem k comportam-se como constantes, permitindo que a complexidade na prática seja avaliada como $O(n)$. No entanto, essa extrema velocidade de processamento cobra um preço arquitetural substancial: o algoritmo não é *in-place*. Ele necessita de uma complexidade espacial de $O(n)$, exigindo a alocação de vetores auxiliares de tamanho idêntico ao conjunto de dados original.

Para demonstrar a rapidez e a sua complexidade espacial de $O(n)$ fizemos uma comparação direta com o quick sort que tem complexidade $O(n \log_2 n)$:

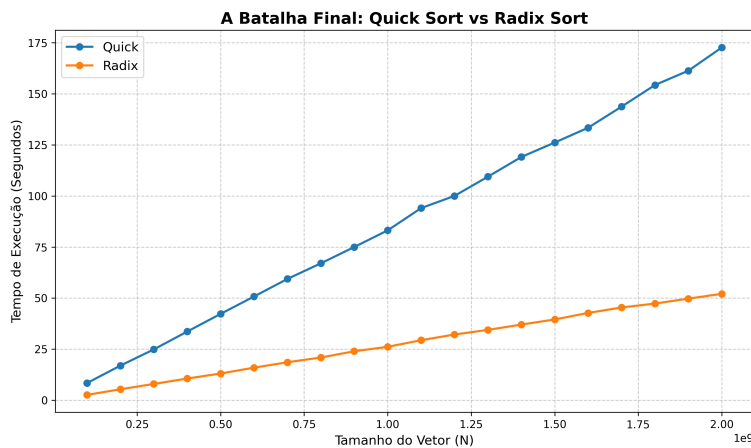


Figura 6: Comparativo de tempo de execução entre Quick Sort e Radix Sort evidenciando a superioridade do modelo não-comparativo em vetores massivos.

Através da análise dos dados experimentais, é possível observar uma progressão clara de eficiência. Para vetores de menor magnitude, nota-se que o Quick Sort consegue se manter competitivo. Esse fenômeno ocorre devido ao *overhead* inicial (custo de preparação) imposto pelo Sistema Operacional

para encontrar e alocar na memória RAM os blocos contíguos massivos necessários para os vetores auxiliares do Radix Sort, além das múltiplas varreduras de contagem.

Contudo, à medida que a ordem de grandeza do vetor N avança substancialmente ao longo do eixo, a inclinação linear da curva do Radix Sort revela a sua verdadeira superioridade matemática. Enquanto a curva do Quick Sort apresenta uma leve curvatura ascendente inerente ao fator logarítmico da sua complexidade $O(n \log_2 n)$, a curva do Radix Sort mantém-se linear e consideravelmente mais baixa, finalizando a ordenação do mesmo volume de dados em um intervalo de tempo notavelmente menor.

É imperativo ressaltar, entretanto, que essa soberania temporal exibida no gráfico depende exclusivamente de um ambiente de hardware favorável. Caso a magnitude do vetor extrapolasse a capacidade física total da memória RAM da máquina de testes, o Sistema Operacional seria forçado a iniciar a paginação em disco secundário. Nessa eventualidade, a curva de tempo do Radix Sort sofreria uma quebra dramática, similar à exaustão de Cache observada no Heap Sort, degradando o seu tempo de execução para patamares inaceitáveis. Como o sistema possuía recursos estritos suficientes para sustentar as cópias, o comportamento $O(n)$ foi validado.

7 Reflexão sobre Uso de IA

- O grupo deverá informar:
- quais ferramentas de IA foram utilizadas;
 - em quais etapas auxiliaram;
 - erros encontrados nas respostas geradas;
 - correções realizadas pelo grupo.

8 Conclusão

Resumo dos principais achados.

9 Referências Bibliográficas